



Southeast University

Department of Computer Science & Engineering

Complex Engineering Project (CEP)

Submitted by,

Md. Abdullah Al Afif

Course Code – CSE345

**Course Title – Information System Design &
Software Engineering**

Submitted to,

Mehedi Ahamed

Lecturer, Department of CSE

Southeast University

Table of Contents

1. Introduction and Scope of Analysis	1
2. Task 1: Problem and Development Method.....	2
2.1 Why This Is a Hard Problem to Define	2
2.1.1 Operational factors.....	2
2.1.2 Technical factors.....	2
2.1.3 Organisation and rules	2
2.1.4 Human behaviour.....	3
2.2 Comparing Possible Development Methods.....	3
2.2.1 Rejected option 1: Waterfall	3
2.2.2 Rejected option 2: Pure Scrum	3
2.2.3 Chosen method: Staged Delivery with an Early Risk Phase	4
2.3 Why the Chosen Method Fits Better.....	4
2.4 Known Weaknesses of the Chosen Method	5
2.5 A Short Note on the Spiral Model	5
3. Task 2: Requirements, Priorities and Schedule.....	6
3.1 How Requirements Were Gathered	6
3.2 Functional Requirements	6
3.3 Non Functional Requirements	7
3.4 Requirements Driven by Fixed Limits.....	8
3.5 A Simple Way to Set Priorities.....	8
3.5.1 The Scoring Applied to Real Requirements	9
3.6 Stakeholder Conflicts and How They Are Solved.....	9
Conflict 1: Company privacy versus full visibility	9
Conflict 2: Worker trust versus audit proof	10
Conflict 3: The old system versus worker workload	10
Conflict 4: Blood safety rules versus a simple workflow	10
3.7 The Phase 1 Schedule	10
4. Task 3: System Design in UML	12
4.1 Approach to the Design	12
4.2 Use Case Diagram	13
4.3 Class Diagram.....	14
4.4 Sequence Diagram.....	15
4.5 Activity Diagram	16
4.6 Design Patterns Used.....	16
4.7 How the Diagrams Fit Together	17
5. Task 4: Testing and Cost Estimation.....	18
5.1 Different Types of Software, Different Risks.....	18
5.2 Testing Plan by Software Type.....	18
5.3 Two Worked Examples	18
5.4 Sharing Testing Effort Fairly Within the Budget	19
5.5 Estimating the Effort Needed: Function Point Analysis.....	19
5.6 Turning Size Into Effort: COCOMO II	19
5.7 An Honest Finding: The Full Scope Does Not Fit in 12 Months	20
5.8 Phase 1 Cost Plan.....	20
5.9 Main Risks to the Estimate	20

5.10 A Range of Possible Outcomes	21
6. Conclusion.....	22
7. References	22

List of Figures

Figure 1: The Phase 1 schedule, made as an Excel Gantt chart	11
Figure 2: Use case diagram showing the main users and what they can do	13
Figure 3: Class diagram showing the main parts of the system	14
Figure 4: Sequence diagram showing a temperature alert and sync	15
Figure 5: Activity diagram showing offline sync and conflict handling	16

List of Tables

Table A: Where each assignment task is answered in this report.....	1
Table 1: What each factor means for planning the project	3
Table 2: Comparing the three methods	4
Table 3: How requirements were gathered from each group	6
Table 4: What the system must do in Phase 1	7
Table 5: Quality goals and how to measure them.....	7
Table 6: Requirements that come from fixed outside limits	8
Table 7: The five scoring questions and their weight	8
Table 8: Scoring applied to selected requirements	9
Table 9: The simple life cycle of a batch of stock.....	12
Table 10: The four layers of the system.....	12
Table 11: The main types of software and what can go wrong.....	18
Table 12: Testing methods used for each software type.....	18
Table 13: How the testing effort is shared out	19
Table 14: Function point count for the Phase 1 system.....	19
Table 15: Phase 1 cost plan.....	20
Table 16: The biggest risks to this estimate.....	21
Table 17: Effort needed for different possible project sizes.....	21

1. Introduction and Scope of Analysis

This report answers the MeghChain brief. MeghChain is a plan to digitise the way vaccines, blood products and essential medicines are tracked and moved across the Sylhet to Meghalaya hill corridor. The corridor has about 240 rural clinics, 9 district hospitals and 3 border quarantine posts spread across 5 districts. This is treated as a real engineering problem that involves health policy, human behaviour and cross border rules, not just a coding task.

Three ideas guide the whole report. First, every design choice is linked to a real detail in the scenario, such as poor network coverage or the old vaccine system that cannot be switched off. Second, when an option is rejected, the reason for rejecting it is written down, because in a problem like this the reasoning matters as much as the final design. Third, two real failures from the scenario are used as the test of success. These are the 40,000 vaccine doses lost in the March 2026 floods, and the smart thermometer pilot that health workers abandoned because they feared surveillance. If a design cannot stop the first failure or avoid the second, it has not succeeded, no matter how advanced it looks on paper.

The report has four main sections, one for each task in the brief. Section 2 looks at the problem and picks a development method. Section 3 builds the requirements, offers a simple way to prioritise them, and lays out the Phase 1 schedule. Section 4 shows the system design using four UML diagrams and explains the design choices in plain terms. Section 5 covers testing and gives a cost and effort estimate.

2. Task 1: Problem and Development Method

2.1 Why This Is a Hard Problem to Define

A simple problem has one clear goal and one way to check if the work is correct. MeghChain does not have this. Different groups want different things from the system. The Ministry of Health wants safety. The funder wants proof for audits. Health workers just want something that helps them and does not get them in trouble. There is no single answer that satisfies everyone, so the problem must be broken down carefully before it can be solved. The main reasons are grouped below, with short codes used later in the report to show where each design decision comes from.

2.1.1 Operational factors

- **OP1. Vaccines cannot wait.** A cold chain break found two weeks later is not a late report. It is a batch of medicine that is already ruined. The whole value of this system depends on how fast it notices a problem, not on how many features it has.
- **OP2. The network cannot be trusted.** Six clinics have patchy mobile coverage and two clinics have none for days at a time in the monsoon season. The app must work fully offline as the normal case, not as an extra feature added later.
- **OP3. Roads can disappear.** Two of the three main roads were underwater for eleven days in March 2026. Delivery plans must be able to change quickly when a route becomes unusable.
- **OP4. Products are not all the same.** Vaccines, blood products and general medicines each have different rules. Treating them as one generic item would break down quickly once blood products are added.

2.1.2 Technical factors

- **TE1. An old system cannot be removed yet.** The 2011 vaccine stock system must keep receiving data until Phase 1 ends. The new system has to send data to it without knowing much about how it works inside.
- **TE2. A second system is outside our control.** NBR customs runs its own old system and it is not connected to anything. Sharing data across the border is politically sensitive, so the design cannot depend on this link working.
- **TE3. Sensor readings must be trusted.** Temperature readings decide whether stock is thrown away or kept. If a reading is wrong or a clock is wrong, the wrong decision could be made, so the data needs to be checked and protected.
- **TE4. Conflicts will happen often.** Because many clinics work offline for days, two people may edit the same record at the same time without knowing it. The system must have a clear rule for solving this, not just hope it rarely happens.

2.1.3 Organisation and rules

- **OR1. No single boss.** The health ministry, the EPI unit, two private companies, the Red Crescent and five district offices are all part of this project. No one person can force a decision, so conflicts must be solved by agreement, not by order.
- **OR2. Money depends on proof.** 65 million US dollars is paid out in stages, and each stage needs an independent audit. If the system cannot produce clear proof of what happened, the project can lose its funding even if the software works fine.
- **OR3. Companies want to keep secrets.** Two private distributors compete with each other. They will not share live stock and delivery data with each other or with the health ministry. The system has to work around this.
- **OR4. Blood has stricter rules.** Blood products need consent records and stronger chain of custody than vaccines or normal medicine, so they need their own set of rules inside the system.
- **OR5. Crossing a border matters.** The corridor crosses an international border. Information that is normal at home can become a sensitive issue at the border post.

2.1.4 Human behaviour

- **HU1. Trust was already broken once.** Health workers in two districts stopped using a smart thermometer pilot because they believed it was used to spy on them. This already happened, so it is a real risk, not a guess.

- **HU2. Honesty needs to be safe.** If a worker who reports a mistake gets blamed for it, workers will stop reporting mistakes. The system must never punish honest reporting, or the data it collects will become false.
- **HU3. Not everyone is used to smartphones.** Workers have different levels of experience with technology and speak Bangla or Sylheti. The app must be simple enough for someone using it for the first time.
- **HU4. Old methods still work sometimes.** Radio calls and informal couriers are not perfect, but they work today. The new system should support these methods, not remove them completely on day one.

Table 1 shows how each of these factors turns into something the project cannot fully plan for in advance.

Code	Factor	What we cannot know at the start	Result for planning
OP2	Weak network coverage	How often each clinic actually has a signal during the monsoon	Must be measured in the field before we commit to a full design
TE1	Old vaccine system	How its data format really works, since it has little documentation	A short research phase is needed before we can estimate this work fairly
TE2	Border customs system	Whether an automatic link will ever be politically allowed	This work must be kept separate so it does not block anything else
OR1, OR3	Many groups with different goals	Where the final agreement on data sharing will land	The requirements will likely change again after the first version is shown
HU1, HU2	Trust of health workers	Whether workers will actually use the app honestly	Only found out by testing with real workers early, not by asking in a survey
OR2	Funder audits	What exact proof the auditor will accept as evidence	The evidence system should be tested against a real audit early on

Table 1: What each factor means for planning the project.

2.2 Comparing Possible Development Methods

Three approaches were looked at closely. These are a step by step plan called Waterfall, a flexible approach called Scrum, and a mixed approach built for this project. Spiral is also mentioned briefly in section 2.5. Each approach is judged against the six unknowns in Table 1, because a method that handles some of them but not others will still cause problems later.

2.2.1 Rejected option 1: Waterfall

Waterfall looks like a good fit at first. The funder pays in stages against a fixed plan, which matches Waterfall well. Blood product rules also favour clear documents that are signed off before work begins. A fixed budget and a fixed twelve month deadline also seem to invite a fixed plan.

It was still rejected, for three clear reasons. First, Waterfall needs the requirements to be known before building starts, but four of the six unknowns in Table 1 simply cannot be known yet. Locking these into a plan too early means the mistakes are found at the very end, when there is no time or money left to fix them. Second, HU1 shows that trust problems only appear once real users try the system. Waterfall does not show anything to real users until the very end, so it has no way to catch this early. Third, OP3 shows that the environment itself can change during the project, since floods can happen again. A fixed plan cannot easily absorb a surprise like that.

2.2.2 Rejected option 2: Pure Scrum

Scrum is good at exactly what Waterfall struggles with. It allows changes, gets feedback from real users early, and can react quickly after a disaster. It was still rejected as the method for the whole project, for three reasons.

- **It does not match how the money is released.** Funding stage payments need a plan that is agreed months in advance. Scrum avoids fixing a plan that far ahead, so it does not fit here.
- **There is no single decision maker.** Scrum needs one Product Owner who can decide what is most important. Under OR1, no single group here can play that role without the others objecting.
- **Some big decisions cannot wait.** Things like how offline data syncs, or how equipment gets delivered to 240 clinics, need a plan made early. Waiting for these to emerge slowly through small sprints would be too risky and too slow.

2.2.3 Chosen method: Staged Delivery with an Early Risk Phase

The method chosen for this project is called Staged Delivery with an Early Risk Phase. It starts with a short phase focused only on reducing risk, followed by three main stages of work. Each stage is tested with real clinics and checked by a funder audit before moving to the next stage. Inside each stage, the team still works in short two week cycles like Scrum. The structure is explained below.

1. **Stage 0, the risk phase (Months 1 to 3).** No features are promised yet. The team studies the old vaccine system, tests the real network coverage, runs early trust building sessions with health workers, and builds one small working example end to end. The Phase 1 plan is then re checked using real evidence instead of guesses.
2. **Stages 1 to 3 (Months 3 to 12).** Stage 1 builds the core cold chain tracking for one pilot district. Stage 2 adds alerts, the link to the old system, and the dashboard, across three districts. Stage 3 adds blood product handling, the border process, and the full rollout to all 240 clinics.
3. **Four checkpoints.** At the end of Stage 0 and after each following stage, the funder checks the evidence and releases the next payment. This is also the point where the plan for the next stage is agreed again.
4. **Short cycles inside each stage.** Work is done in two week cycles with regular testing and feedback from the pilot district.
5. **A separate track for things outside our control.** Work that depends on another authority, such as NBR customs, is kept on its own track so it never blocks the main schedule.

Table 2 compares the three methods against the six challenges from Table 1.

What matters	Waterfall	Pure Scrum	Chosen method
Coping with unknown network coverage (OP2)	1	4	5
Coping with the unclear old system (TE1)	2	3	5
Keeping blocked work separate (TE2)	1	3	5
Catching trust problems early (HU1, HU2)	1	5	5
Matching the funder payment stages (OR2)	5	2	5
Planning equipment for 240 sites early	4	2	4
Coping with another disaster mid project (OP3)	1	5	4
Protecting the core design early (TE4)	4	2	5
Overall result	Poor fit	Partial fit	Best fit

Table 2: Comparing the three methods (1 is a poor fit, 5 is a strong fit).

2.3 Why the Chosen Method Fits Better

The strongest reason for the chosen method is that this project has two needs that pull in different directions. The funder needs a plan agreed ahead of time before releasing money. But the real world here only reveals its problems after work has started, such as how bad the network really is, or whether workers trust the app. Waterfall serves the first need and fails the second. Scrum serves the second need and fails the first.

The chosen method solves this by moving the point of commitment. Instead of committing to the whole year at once, or refusing to commit at all, the team commits to about three months at a time. This is short enough that the plan reflects what has just been learned, and long enough for the funder to see a clear plan ahead. Stage 0 exists so that the first big commitment only happens after the four biggest unknowns have been tested for real. About one quarter of the year is spent on this before any feature ships, and this is worth it because guessing the size of the old system work, without ever having looked at it, would put the whole schedule at risk.

This also solves the trust problem in the right order. Trust building with the community starts in Month 1, and the first version shown to real clinics in Month 6 only includes features the community has already reviewed. Waterfall would delay any contact with real users. Scrum, on its own, might ship a location tracking feature early just because it looks useful, without checking if it repeats the mistake that ended the earlier pilot.

2.4 Known Weaknesses of the Chosen Method

Three weaknesses are accepted on purpose. First, four funder checkpoints in one year take real time to prepare for, so the plan turns the audit evidence into something the system produces automatically, rather than a manual report written separately. Second, a mid stage discovery normally has to wait until the next checkpoint, unless it is a safety issue, in which case it can be raised immediately. Third, everything depends on Stage 0 being done properly. If the early research and testing is rushed, this method becomes Waterfall with extra steps. This is the biggest risk in the whole plan, which is why Stage 0 is only considered finished when there is real evidence, such as a working test version and real network data, not just a set of documents that were approved in a meeting.

2.5 A Short Note on the Spiral Model

The Spiral model was also considered, because it focuses on solving risk first, which matches Stage 0 closely. It was not chosen as the main method because its stages are defined by how much risk is left, not by what the funder can measure as a finished result. The funder pays for results like three districts being live, not for a general reduction in risk. Because of this, the chosen method borrows the risk first idea from Spiral for Stage 0, but uses clear delivery stages for the rest of the project.

3. Task 2: Requirements, Priorities and Schedule

3.1 How Requirements Were Gathered

Different methods were used for different groups, because each group is comfortable sharing different kinds of information. A private company will not talk openly about its delivery routes if a competitor is in the room. A health worker will not admit to a shortcut in front of an official visitor. Table 3 shows the method chosen for each group and why.

Group	Method used	Why this method fits this group
Health workers and clinic staff	Watching real work at clinics, small group design sessions, and a private way to raise concerns	People often hide problems in interviews if they fear blame or surveillance. Watching real work shows the true picture.
Health ministry and EPI unit	Reading their existing reports and holding planning workshops	Most of their needs are already written down in current reports. The main task is understanding the old system.
Private distributors	Private one to one meetings, never with competitors in the same room	A shared meeting would make both companies give the safest, least useful answers.
Funder and audit team	A planning workshop and a practice audit walkthrough	The real requirement is what the auditor will actually accept as proof, which is best found by testing it directly.
NBR customs authority	Formal letters and watching the process at the border post	Contact is limited by official rules, so watching the real process gives more accurate information than asking.
Red Crescent and civil surgeons	Workshops that replay the March 2026 flood event	Their real needs only become clear when discussing an actual disaster, not a general question.
Blood bank staff	Reviewing the legal rules directly with them	Their requirements mostly come from law, so the task is matching the system to the law, not asking for preferences.

Table 3: How requirements were gathered from each group.

3.2 Functional Requirements

These are the things the system must be able to do. Each one has an ID and is linked to the factor in Section 2.1 that makes it necessary.

ID	The system shall	Main group asking for it	Linked to
FR1	Let a worker record stock received, issued, moved or thrown away, even with no internet	Health worker	OP2
FR2	Read temperature data from sensors and detect when it goes out of range	EPI unit	OP1, TE3
FR3	Raise an alert within 5 minutes of a problem and block that batch from being used	EPI unit	OP1
FR4	Let an officer confirm whether a flagged batch is still safe or must be thrown away	District officer	OP4
FR5	Keep a permanent record of every time a batch changes hands	Auditor	OR2
FR6	Record and check donor consent for blood products	Blood bank	OR4
FR7	Plan and track deliveries across multiple stops and vehicles	Distributor	OP3
FR8	Let an officer change a delivery route if a road becomes blocked	Civil surgeon	OP3
FR9	Send saved offline data to the server whenever a connection appears, and solve conflicts safely	All field workers	OP2, TE4
FR10	Send a short alert by text message when there is no internet at all	Health worker	OP2
FR11	Show distributor delivery capacity only as a general range, never exact numbers	Distributor	OR3
FR12	Show a dashboard that only displays what a person is allowed to see	Health ministry	OR1
FR13	Send stock updates to the old vaccine system automatically every night	EPI unit	TE1
FR14	Produce a full audit report on request, ready for an independent check	Funder	OR2
FR15	Create a delivery paper for the border post that works without any network link	NBR, quarantine officer	TE2, OR5
FR16	Let an administrator manage who can see and do what, with every change recorded	Auditor	OR1, OR3

ID	The system shall	Main group asking for it	Linked to
FR17	Flag any sensor that has not been calibrated recently, so its readings are treated as unsure	EPI unit	TE3
FR18	Show the health worker a simple, non blaming list of anything that needs fixing after a sync	Health worker	HU2

Table 4: What the system must do in Phase 1.

3.3 Non Functional Requirements

These describe how well the system must work, not what it must do. Each one includes a way to measure if it has been met, because a quality goal that cannot be measured cannot be checked at an audit.

ID	Quality	Requirement and how to check it	Linked to
NFR1	Works offline	The app must work fully for at least 14 days with no internet and lose no data when it reconnects	OP2
NFR2	Fast detection	A problem must be shown to the local user within 5 minutes, with or without internet	OP1
NFR3	Fast alert to officers	If a connection exists, the district officer is alerted within 30 minutes. If not, within 30 minutes of the next connection	OP1, OP2
NFR4	Data cannot be changed	Safety and custody records can only be added, never edited or deleted, by anyone including administrators	OR2, OR4
NFR5	Company data stays private	No distributor can see another distributor's exact numbers. Shared data is always grouped, never shown for a single company	OR3
NFR6	Ready for audit	A full audit report can be produced and checked within 24 hours of being requested	OR2
NFR7	Easy to use	A worker with no smartphone experience can record stock, issue stock and log a temperature problem after 90 minutes of training, with 95 percent success	HU3
NFR8	Clear about data use	The app must show the user, in Bangla, exactly what data it holds and what it will send, at any time	HU1
NFR9	Works on slow networks	A full day of one clinic's activity must sync in under 90 seconds on a slow 2G connection	OP2
NFR10	Works on low battery	The device must last a 12 hour work day without charging, and never lose data if power is lost mid save	OP2
NFR11	Stays online	The main system should be available 99.5 percent of the time each month, and local areas keep working during a central outage	OR2
NFR12	Local language	The whole app is available in Bangla, with correct Sylheti terms for cold chain words, checked by real users	HU3
NFR13	Secure	Data on the phone is encrypted, every action is signed, and the app passes an outside security test before full rollout	OR2, OR4

Table 5: Quality goals and how to measure them.

3.4 Requirements Driven by Fixed Limits

These requirements exist only because of a fixed rule or condition outside our control. They are not quality goals that can be adjusted. They are hard limits that must be respected exactly as they are.

ID	The fixed limit in the scenario	What the system must do because of it
CR1	The old vaccine system cannot be turned off before Phase 1 ends	The system must keep sending updates to the old system every day, but must never read it back as the source of truth
CR2	NBR customs runs a separate system with no live connection	A border delivery paper must be created without needing any live link to NBR
CR3	Two clinics have no signal for days, six have patchy signal	No safety feature can require internet. The app should try data, then a low power network, then text messages, then work fully offline
CR4	Distributors will not share live stock or route data	The system only accepts general delivery capacity ranges, never exact company numbers
CR5	Blood products need stricter consent and custody rules	Blood handling must follow its own separate rules from vaccines and medicine

ID	The fixed limit in the scenario	What the system must do because of it
CR6	Funding is paid in stages against independent audits	The system must be able to produce solid proof for each funding stage as it happens, not after the fact
CR7	A past pilot failed due to fear of surveillance	Location data must be general, not exact, must be visible to the user, and must never be used to judge individual workers
CR8	6.2 million US dollars and 12 months for Phase 1	The work must be limited to what fits this budget and time, with equipment delivery planned to happen before the software rollout at each site
CR9	Five districts under one shared authority	Every user and record must be tied to a specific role and district, not just a general permission

Table 6: Requirements that come from fixed outside limits.

3.5 A Simple Way to Set Priorities

MoSCoW was the first option considered, but it was not used as it is normally taught. MoSCoW only ranks items in order. It cannot show that two items are both marked as Must Have but one is far more urgent. It also has no way to protect against a feature that looks useful but repeats the trust mistake from the earlier pilot, since a feature like that could easily be marked Must Have by mistake.

Instead, a simple scoring method was built for this project, called the MeghChain Priority Score. Each requirement is scored from 1 to 5 on five simple questions, and the scores are combined using weights, as shown in Table 7.

Short name	Question asked	Weight
Safety	If this is missing for one more month, how bad could the harm to a patient be?	0.30
Audit	Does a funding payment or a legal rule depend on this being shown clearly?	0.20
Trust	Does this make health workers more willing, or less willing, to use the system honestly?	0.20
Build order	How much other work depends on this being built first?	0.15
Disaster value	How much worse would things get without this during a flood or outage?	0.15

Table 7: The five scoring questions and their weight.

The final score is Safety times 0.30, plus Audit times 0.20, plus Trust times 0.20, plus Build order times 0.15, plus Disaster value times 0.15. A score of 4.00 or higher goes into Stage 1. A score from 3.20 to 3.99 goes into Stage 2. A score from 2.40 to 3.19 goes into Stage 3. Anything lower is left for Phase 2. Safety carries the most weight on its own, but Audit and Trust combined carry even more weight than Safety, because a system that is safe but gets defunded, or refused by its own users, ends up helping nobody at all.

Two extra rules sit on top of the score, because a number alone cannot capture everything that matters here.

- **Rule A, legal items always go in on time.** Any requirement needed to follow the law must be included in the same stage as the feature it protects, no matter what its score is. For example, blood consent tracking must go in with blood handling, not later.
- **Rule B, trust items cannot be skipped.** Any requirement that the community review group flags as feeling like surveillance cannot be released unless the matching privacy feature ships in the same stage. This directly protects against repeating the smart thermometer failure.

3.5.1 The Scoring Applied to Real Requirements

Table 8 shows the scoring applied to a sample of requirements, chosen because they show cases where this method gives a different answer than a normal priority list would.

Req.	Safety	Audit	Trust	Build order	Disaster	Score	Stage	Note
FR2	5	4	3	5	5	4.40	Stage 1	Detecting a problem is the whole point of the system
FR9	4	4	4	5	5	4.30	Stage 1	Not exciting to look at, but everything else depends on it
FR3	5	3	3	3	5	4.05	Stage 1	The alert half of the safety chain

Req.	Safety	Audit	Trust	Build order	Disaster	Score	Stage	Note
FR5	4	5	2	4	3	3.75	Stage 1	Rule A places this early since audits need it right away
NFR8	2	2	5	2	2	2.80	Stage 1	Rule B moves this up. This directly avoids the old surveillance fear
FR13	2	5	2	4	2	3.00	Stage 2	No direct safety value, but the audit needs it
FR12	2	4	3	2	3	2.85	Stage 2	Dashboards look impressive but are not the most urgent item
FR6	4	5	3	3	2	3.65	Stage 3	Rule A ties this to when blood handling itself begins
FR8	3	2	3	2	5	3.10	Stage 3	Mostly needed for the flood and disaster case
FR15	2	4	2	2	3	2.65	Stage 3	The paper export ships on time, the live NBR link does not

Table 8: Scoring applied to selected requirements.

3.6 Stakeholder Conflicts and How They Are Solved

The four conflicts below are real trade offs. Helping one group in each case makes things harder for another group. Each one is solved with a clear decision, and the cost of that decision is stated honestly rather than hidden.

Conflict 1: Company privacy versus full visibility

Health officials want to see exactly where all stock is so they can respond quickly during a disruption. Distributors will not share exact stock and delivery numbers, since it would expose their business to competitors. These two needs are directly opposed.

Solution. Distributors only publish a general delivery range per district and time window, not exact numbers. Full detail is only kept for stock that the Trust itself owns, such as vaccines and blood.

Cost. During a flood, the Trust will know a distributor has some capacity in an area, but not exactly which truck holds what. A phone call may still be needed to act on it. This is accepted because forcing full disclosure would likely cause the distributors to give false or incomplete data instead, which would be worse.

Conflict 2: Worker trust versus audit proof

Auditors naturally want to know exactly who did what, when, and where. But tracking individual workers closely is the same thing that made health workers abandon the smart thermometer pilot.

Solution. Records are linked to a role and a clinic, such as "store keeper at Clinic 42", not to a named person, in normal use. The real identity is only kept in a protected audit trail and can only be revealed through a formal, recorded request. Location is recorded at the clinic level, not as an exact GPS trail, and the app always shows the worker what data it holds.

Cost. Investigating a specific problem takes an extra formal step and is slower. This is accepted because the March 2026 losses were caused by missed warnings, not by any individual doing something wrong, so constant personal tracking would cost trust without adding real safety value.

Conflict 3: The old system versus worker workload

The EPI unit needs the old system to keep receiving data. If health workers had to enter every record twice, once in the new app and once in the old system, it would double their workload and the two systems would soon disagree with each other.

Solution. Workers enter data only once in the new system. A background process automatically sends the needed updates to the old system every night. The old system is never treated as the correct source of truth.

Cost. The EPI unit loses the ability to correct records directly in the old system during Phase 1. This is accepted because building a two way link with a 2011 system that is already being retired would waste budget that could be better spent elsewhere.

Conflict 4: Blood safety rules versus a simple workflow

Blood products need extra steps for consent and custody. But the app also needs to stay simple enough for a first time smartphone user, and adding blood specific steps to every workflow would make it harder for everyone.

Solution. The extra steps only appear when handling blood products, and only trained blood custodians see that workflow. Vaccine and medicine workflows stay short and simple.

Cost. Two separate workflows have to be built, trained and tested instead of one. This adds real cost, which is reflected in the Stage 3 budget in Section 5.

3.7 The Phase 1 Schedule

Figure 1 shows the twelve month schedule. It follows the scoring bands from Section 3.5 directly. Stage 1 covers everything scored 4.00 or above. Stage 2 covers 3.20 to 3.99. Stage 3 covers the rest, plus the legal items from Rule A. The schedule was built as a working Excel file where every bar and every effort total is calculated by a formula, so any change to the plan automatically updates the totals.

A few choices in the schedule are worth explaining. Stage 0 takes three full months and delivers no visible feature, for the reasons given in Section 2.3. The community trust programme runs for all twelve months, not just at launch, because trust needs to be maintained the whole time, not built once and forgotten. Equipment such as sensors and coolers starts arriving from Month 1 and finishes by Month 8, ahead of the software rollout, because a clinic without working equipment cannot use the app no matter how good it is. The full rollout to all clinics is placed in the last three months, which is also when the most staff are working at once.

Staffing is kept low in the first three months, at around 12 people, and rises to about 26 people during the final rollout. This is a deliberate choice. If Stage 0 finds a serious problem, such as the old system being far more complex than expected, very little has been spent yet, so the plan can still be safely adjusted. In total, the plan uses 251 person months, made up of 114 person months of software work and 137 person months of field work, training, equipment and management. Section 5.6 checks this total against a separate cost estimate.

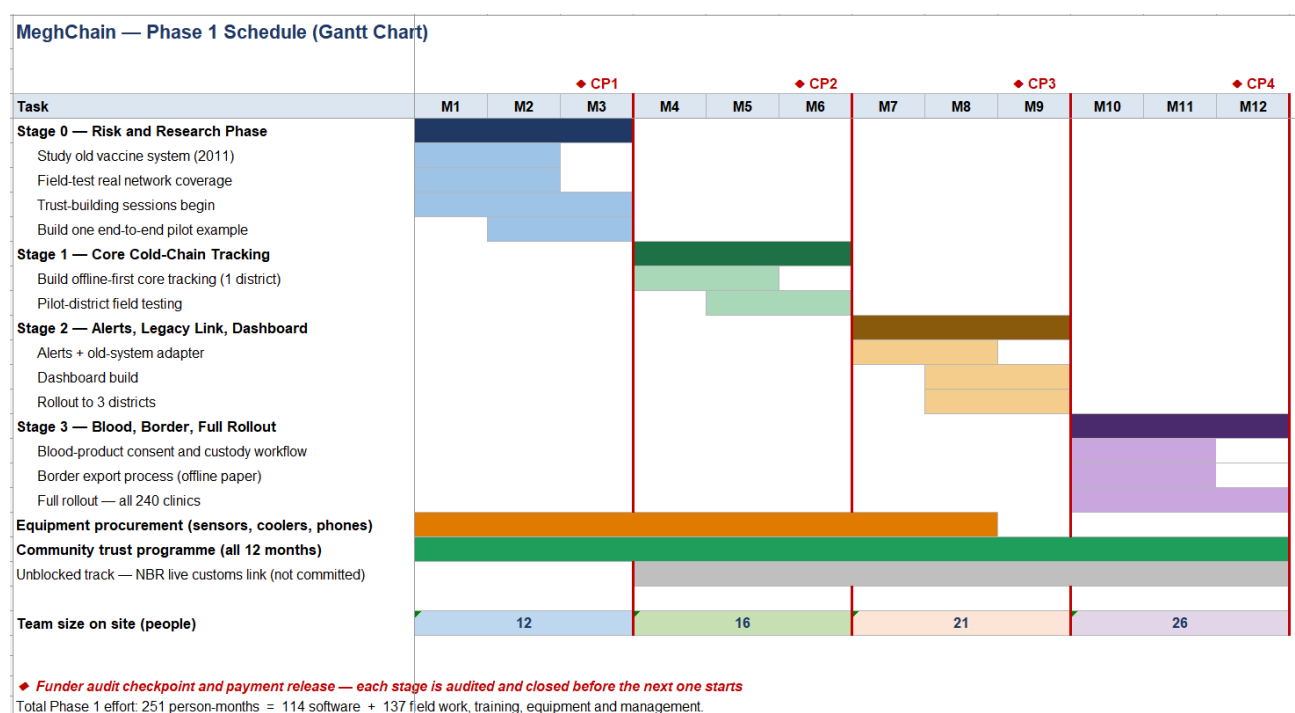


Figure 1: The Phase 1 schedule, made as an Excel Gantt chart. Each half of a month is one column. Dark blue bars are stage totals, light blue bars are individual tasks, and red bars are the funder checkpoints.

The four checkpoints shown in the schedule, at the end of Stage 0 and after each of the three later stages, are what let this plan work for both sides. The funder gets a clear plan to check every few months. The project still gets a chance to adjust the plan as it learns more about the real conditions in the corridor.

4. Task 3: System Design in UML

4.1 Approach to the Design

Four UML diagrams are used in this section: a use case diagram, a class diagram, a sequence diagram and an activity diagram. These four were chosen because together they answer the most important questions about MeghChain in a way that is easy to explain out loud. The use case diagram shows who uses the system and what they can do. The class diagram shows what the system is made of. The sequence diagram shows how a single alert travels through the system step by step. The activity diagram shows what happens when data from an offline clinic finally reaches the server. Between them, these four diagrams cover both the structure of the system and its behaviour, which is what the assignment asks for.

Two ideas from earlier designs are kept in this section, but shown as short tables instead of extra diagrams, so the design stays easy to follow. Table 9 shows the simple life cycle of a batch of stock. Table 10 shows the four main layers of the system. This keeps the important design thinking in the report without adding diagrams that would be harder to explain in a short amount of time.

Status	What it means	What can happen next
Normal	The batch is within safe temperature range and can be used	Moves to Flagged if a bad reading comes in, or is issued to a patient
Flagged	A bad temperature reading was found and the batch is blocked from use	An officer reviews it and moves it to Safe or Unsafe
Safe	An officer has checked the flagged batch and confirmed it can still be used	Can be issued to a patient like a normal batch
Unsafe	An officer has confirmed the batch must be thrown away	The batch is recorded as discarded and cannot be used again

Table 9: The simple life cycle of a batch of stock.

One safety rule sits on top of this table. If a flagged batch has not been checked by an officer within 72 hours, it is automatically moved to Unsafe rather than left waiting. This matters because a clinic can be cut off for many days at a time, and a batch with no internet connection to reach an officer must never be treated as if it were approved.

Layer	What sits here	Why it is separate
Field layer	The phone app used by health workers at each clinic	This is the only layer that must work with no internet at all
District layer	A small local server at each district hospital	District hospitals usually have better signal than nearby clinics, so this layer lets nearby clinics sync locally even if the main server is unreachable
Central layer	The main server, the dashboard, and the audit records	This is where all the core rules and reports live
Connector layer	Small pieces of code that talk to the old vaccine system, the border system, and the distributors	Keeping these separate means that if an old system changes or is removed later, the rest of the system does not need to change with it

Table 10: The four layers of the system.

4.2 Use Case Diagram

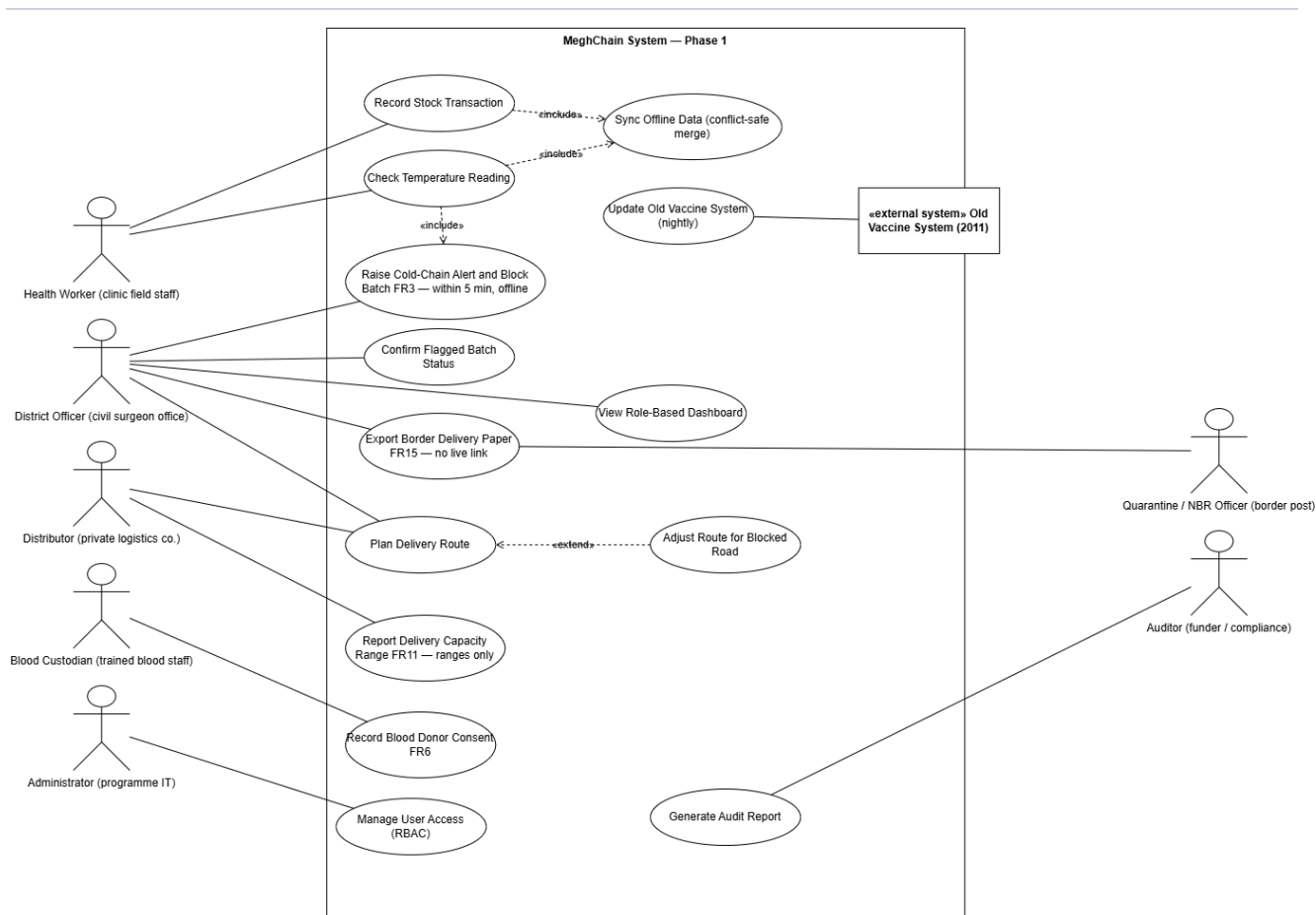


Figure 2: Use case diagram showing the main users and what they can do.

What this diagram shows

This diagram shows the four main types of people who use the system, and the old vaccine system as an outside system. The health worker records stock and checks temperatures. The district officer plans deliveries and views the dashboard. The distributor also plans deliveries but does not see private health data. The auditor generates reports for the funder.

Two include arrows are the most important part of this diagram. Recording stock and checking temperature both include syncing data. This means syncing is not treated as a separate extra step. It is built into the definition of these tasks, because a health worker's work is not finished until it reaches the rest of the system, even if that only happens hours or days later.

4.3 Class Diagram

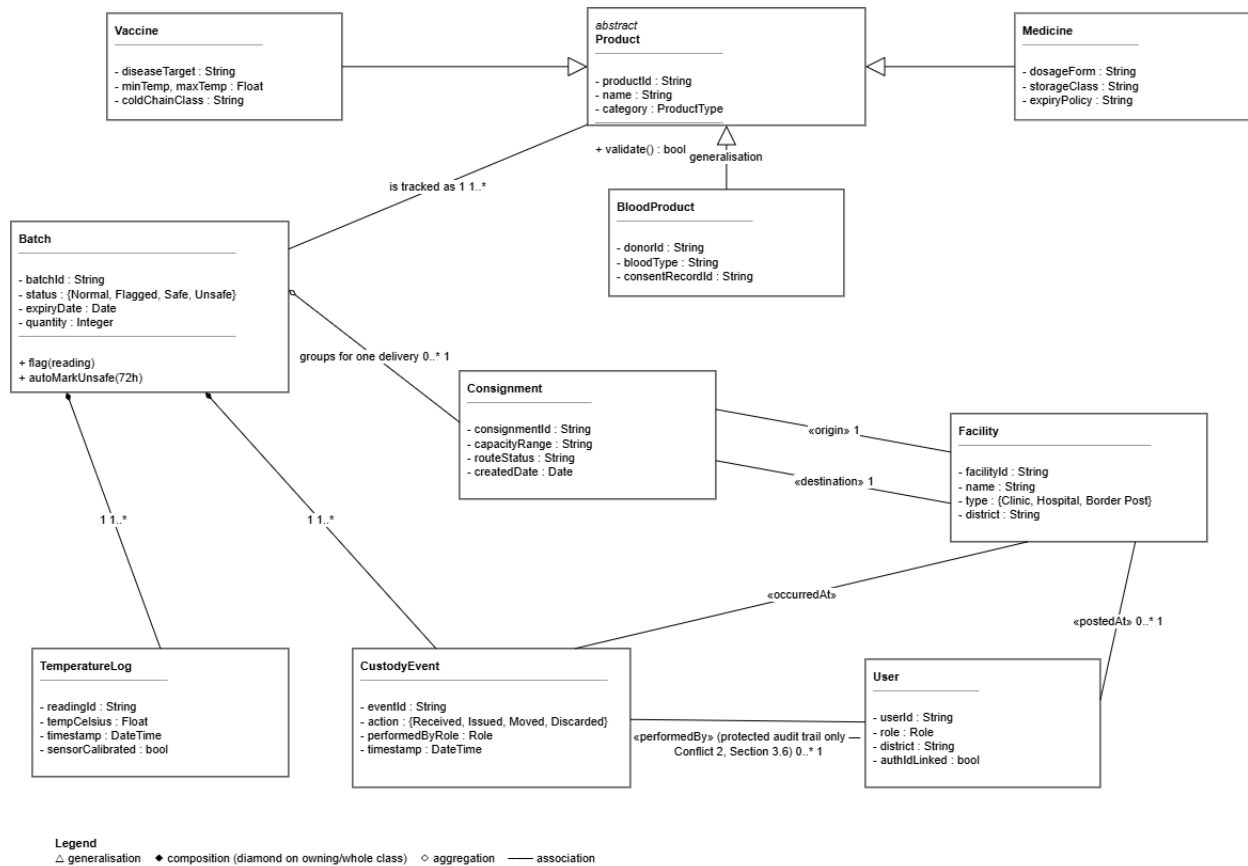


Figure 3: Class diagram showing the main parts of the system and how they connect.

What this diagram shows

Product is shown as a general type with three specific kinds underneath it: Vaccine, Blood Product and Medicine. This matters because blood products carry extra information, such as donor ID and consent, that vaccines and medicine do not need. Keeping these as separate classes, instead of one big class with many unused fields, keeps the design honest about the fact that blood follows different rules.

A Batch is linked to many Temperature Log entries and many Custody Event entries. This means the full history of a batch, including every time it changed hands, is always kept, rather than just storing a single current status. This history is exactly what an auditor needs to see, and it is also what allows the 72 hour safety rule in Table 9 to work correctly.

A Consignment groups many batches together for one delivery and links to two facilities, the starting point and the destination. The User class carries a role and a district, which supports Conflict 2 from Section 3.6, since access is based on a role and a district rather than a single named account.

4.4 Sequence Diagram

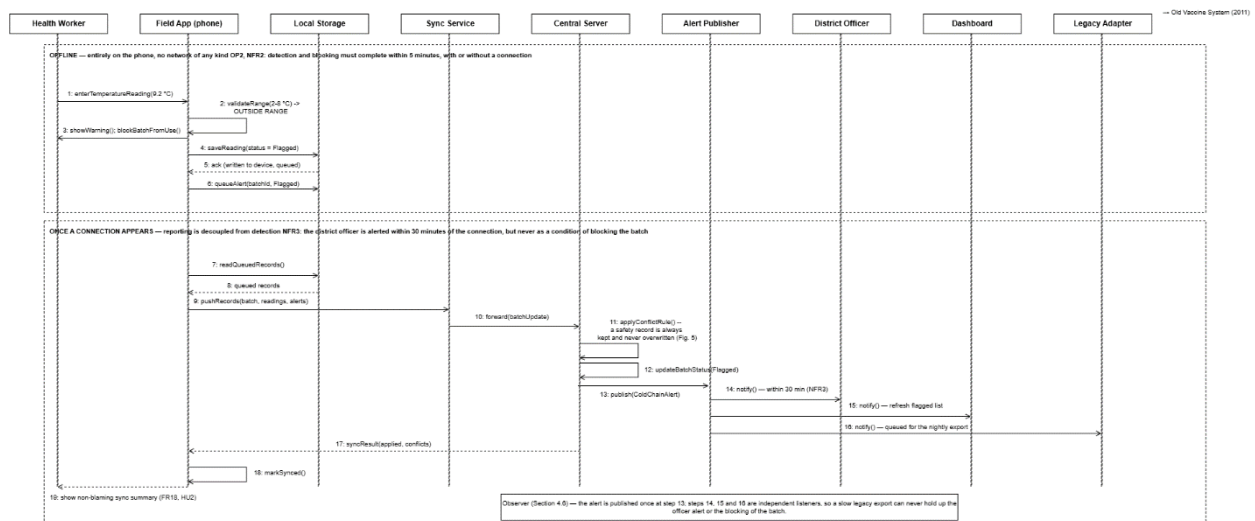


Figure 4: Sequence diagram showing how a temperature alert is raised and later synced.

What this diagram shows

This diagram follows one temperature reading from the moment a health worker enters it, to the moment a district officer is alerted. The first six steps all happen on the phone itself, with no network needed at all. If the reading is unsafe, the app immediately warns the worker and blocks the batch. This step cannot wait for an internet connection, because the whole point of this feature is to catch problems before more damage is done. This directly matches what should have happened during the March 2026 floods.

The second half of the diagram only happens once a network connection appears. The saved records are sent through a Sync Service to the Central Server, which updates the batch status and only then sends an alert onward to the district officer. Splitting the diagram this way shows clearly that detecting a problem and reporting a problem are two separate steps, and the first one must never depend on the second.

4.5 Activity Diagram

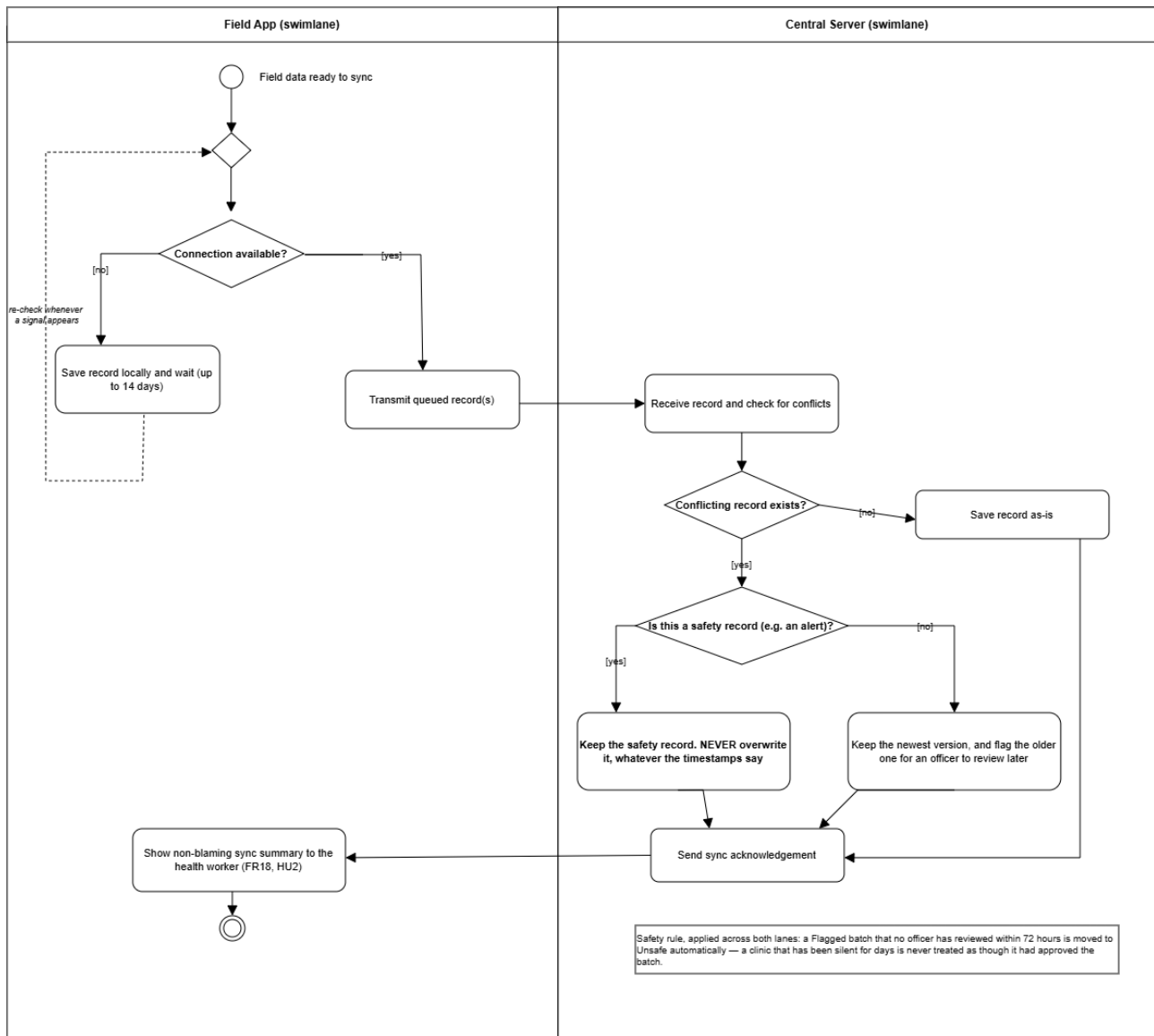


Figure 5: Activity diagram showing how offline data is synced and how conflicts are handled.

What this diagram shows

This diagram has two lanes, one for the Field App and one for the Central Server. The Field App always checks for a connection first. If there is none, it simply saves the data locally and waits, which is the normal case for many clinics in this corridor, not a rare exception.

Once the data reaches the Central Server, it is checked for conflicts. The most important decision in this diagram is the second question, which asks if the record is a safety record such as an alert. If it is, the rule is simple and firm: the safety record is always kept and never overwritten. If it is not a safety record, such as a routine stock count, the newest version is kept but it is also flagged for an officer to check later. This two path rule exists because treating every kind of data the same way would risk a real safety alert being quietly overwritten by an unrelated stock update that happened to arrive later.

4.6 Design Patterns Used

Two design patterns are used in the design shown above. Each one solves a specific problem from the scenario, and each one can be pointed to directly inside the diagrams already shown.

Pattern 1: Adapter

Problem it solves. The old vaccine system must keep receiving data, but the rest of the system should not need to understand its internal format.

How it is used. A small adapter component sits between the Central Server and the old system. It takes the normal data from MeghChain and converts it into the format the old system expects, then sends it automatically every night. This is the "Update Old Vaccine System" use case shown in Figure 2, and it is also why the old system is never queried directly anywhere else in the design.

Why this option and not a simpler one. A simpler option would be to let the main system read and write the old system's format directly. This was rejected because the old system will eventually be shut down. If the whole design had learned its shape from the old system, removing it later would mean rebuilding large parts of the software. With an adapter, removing the old system later is as simple as deleting one small component.

Pattern 2: Observer

Problem it solves. One single temperature alert needs to reach several different places at once: the health worker's screen, the district officer, the dashboard, and the old system update. As the project grows, even more places may need to hear about the same alert.

How it is used. When the alert happens, it is announced once, and each interested part of the system listens for it and reacts on its own. This matches the way the Central Server in Figure 4 forwards the alert onward without needing to call each destination directly one by one.

Why this option and not a simpler one. A simpler option would be to call each destination directly from the alert code itself. This was rejected because it would mean that if one destination, such as the slow and sometimes unreliable old system, was delayed, it could delay or break the most important part of the process, which is blocking the unsafe batch. Keeping the alert and its listeners separate means the most urgent action, blocking the batch, can never be held up by a less urgent one.

4.7 How the Diagrams Fit Together

The four diagrams describe one single system, not four separate ones. The use cases in Figure 2 are carried out in detail by the steps shown in Figure 4. The Batch, Temperature Log and Custody Event classes shown in Figure 3 are exactly the objects being updated at each step of Figure 4 and Figure 5. The batch status values in Table 9 are the same values that Figure 3's Batch class stores and that Figure 4's Central Server changes. Keeping this consistent across all four diagrams, and across the supporting tables, was a deliberate choice so that the whole design can be explained as one connected story rather than four separate pictures.

5. Task 4: Testing and Cost Estimation

5.1 Different Types of Software, Different Risks

MeghChain is not one single piece of software. It is made up of a few different types of software, and each type fails in a different way. Testing effort should match the risk of each type, not be spread evenly across all of them.

Type	How it usually fails	What happens if it fails in real use
A. Field app on the phone	Data is lost or corrupted during a long period offline, or the app is too hard to use	Records from an outage are lost, or workers give up and go back to paper
B. Sync and conflict handling	A record is merged the wrong way, or the same update is applied twice	A safety alert could be silently overwritten by an unrelated stock update
C. Old system and border connectors	Data sent to the old system is rejected or silently fails	The old system falls out of sync without anyone noticing, and an audit fails
D. Sensors and temperature readings	A sensor reads the wrong value or the wrong time	Good stock is wrongly thrown away, or a real problem goes unnoticed
E. Central server, dashboard and audit trail	Someone sees data they should not be allowed to see	A distributor could leave the programme, or an audit could fail

Table 11: The main types of software and what can go wrong with each.

5.2 Testing Plan by Software Type

Type	Main testing methods	Why this fits
A	Testing on real phones with different screen sizes and network types. A 30 day offline test where the app is never given a connection. Real health workers trying the app for the first time	The 14 day offline requirement cannot be checked any other way. Real user testing is the only way to catch confusing screens
B	Automatic tests that check the merge rules with many random combinations of data. Tests that copy real network drop outs and delays	The merge logic is the most important and the riskiest part of the whole system, so it needs the widest possible testing
C	Comparing output files against a known correct example file. Testing what happens when the old system rejects or ignores data. A daily automatic check that compares both systems	There may be no test copy of the old system available, so testing against a known correct file is the safest option
D	Testing sensors against a proper thermometer in a lab. Testing what happens when a sensor gives a clearly wrong reading	The readings decide whether valuable stock is thrown away, so they must be proven accurate, not just assumed to be correct
E	Testing that each role can only see what it should see. An outside security check before the full rollout begins	A privacy mistake here could cause a distributor to leave the programme or cause an audit to fail completely

Table 12: Testing methods used for each software type.

5.3 Two Worked Examples

Testing the temperature limit. The safe range for a vaccine is 2 to 8 degrees. Test values are chosen right at the edges, such as 1.9, 2.0, 2.1, 7.9, 8.0 and 8.1 degrees. These edge values matter the most, because a small mistake at exactly 8.0 degrees could either cause too many false alarms or miss a real problem completely.

Testing the batch status changes. Using Table 9 directly, tests are written to check every allowed change of status, and also to check that forbidden changes are correctly blocked. For example, a test tries to move a batch straight from Unsafe back to Normal, and checks that the system correctly refuses to allow it. Testing what the system correctly refuses to do is just as important as testing what it correctly allows.

5.4 Sharing Testing Effort Fairly Within the Budget

With a fixed budget and a fixed year, it is not possible to test every part of the system equally. Testing time is instead shared out based on how much damage a mistake in that area could cause, not based on how much code that area contains.

Type	Share of testing time	Reason
A. Field app	30 percent	If workers do not trust or cannot use the app, the whole project fails, no matter how correct the code is
B. Sync and conflict handling	25 percent	The highest risk area, but also one that can be tested mostly with automatic tests once it is built
C. Old system and border connectors	18 percent	The daily automatic comparison check gives ongoing safety at very low extra cost
D. Sensors	12 percent	One shared testing lab can cover the whole programme, so full coverage does not need to be expensive
E. Central server and dashboard	15 percent	Effort is focused mainly on one outside security check right before the full rollout

Table 13: How the 33 person months of testing effort are shared out.

5.5 Estimating the Effort Needed: Function Point Analysis

Function Point Analysis was chosen to estimate the size of the system before any code is written. This method was chosen instead of story points because, at this early stage, there is no development team yet with a known working speed, and the funder needs a size estimate that does not depend on which people end up doing the work. Story points are still used later, once Stage 1 begins and a real team speed is known.

Type of item	Simple	Medium	Complex	Points
Data stores kept inside the system	4	6	2	118
Data stores from outside systems	2	2	1	34
Ways data is entered into the system	6	12	6	102
Ways data is shown or sent out	4	9	5	96
Ways data is looked up or searched	5	8	3	65
Total function points				415

Table 14: Function point count for the Phase 1 system.

A general adjustment is then applied for factors such as how spread out the system is across many sites, and how much the system needs to talk to other systems. This raises the total from 415 to about **469 adjusted function points**.

5.6 Turning Size Into Effort: COCOMO II

Function points measure size, not effort. The COCOMO II method is used to turn the size into an effort estimate in person months. The 469 function points are converted into roughly 22,000 lines of code, based on a typical rate for the programming languages planned for this project.

Several factors raise the difficulty of this project above a normal software project. There is no similar system that already exists for this exact corridor. The team is made up of five different organisations, including two competing companies, which makes coordination harder. The reliability needed is very high, since mistakes could cause real harm to patients. Taking all these factors into account, the full scope as described in the brief comes out to about **168 person months** of effort, with a normal, uncompressed schedule of about **18.4 months**.

5.7 An Honest Finding: The Full Scope Does Not Fit in 12 Months

This is the most important result from the estimate, and it is not a positive one. The math shows that the full system, exactly as described in the brief, needs about 18.4 months under normal conditions. Squeezing that down into the 12 months allowed by Phase 1 would need the schedule to run at 65 percent of its normal length. This is beyond what is considered achievable by adding more people to a project, since extra people also mean more time spent on communication rather than actual building.

Because of this, the honest answer is not to promise the full scope in 12 months. Instead, Phase 1 is planned around a smaller, focused core, made up of the Stage 1 and Stage 2 items from Section 3.5, plus the required blood consent features and a simple offline border export. This smaller core needs about **87 person months** of effort and about **15.0**

months under normal conditions. Compressing this smaller core to fit 12 months is a stretch, but it is within a realistic and achievable range, unlike compressing the full original scope.

This smaller core estimate of about 97 person months, once the schedule compression is included, can be compared to the 114 person months already built into the Section 3.7 schedule from a completely different method, based on named tasks and staff numbers. These two numbers, produced in two different ways, are close to each other, which gives good confidence that the plan in Section 3.7 is realistic rather than optimistic.

5.8 Phase 1 Cost Plan

Software work is costed at 4,200 US dollars per person month, based on a team mostly based in Dhaka. Table 15 shows the full cost plan, using the 114 person month software figure from the working schedule.

#	Cost item	Basis	US Dollars
1	Software building and testing	114 person months at 4,200 dollars each	478,800
2	Outside review and security testing	Three review points plus one outside security test	165,000
3	Cold chain equipment	Sensors, coolers and containers for 252 sites	1,620,000
4	Field phones and solar chargers	620 units including spares	226,000
5	Network connections	Low power network, satellite link for two sites with no coverage, and data plans	505,000
6	Community trust programme and training	Twelve month programme for all districts	592,000
7	Old system and border connector work	Research, building and testing the two connectors	285,000
8	Hosting and backup systems	Central system plus nine district servers for one year	174,000
9	Project management and reporting	Full year of programme management and audit reporting	462,000
10	Storage upgrades	Cold rooms at nine hospitals and three border posts	820,000
	Subtotal		5,327,800
11	Extra safety margin, 15 percent	Covers the main risks listed in Table 16	799,170
	Phase 1 total	About 99 percent of the 6.20 million limit	6,126,970

Table 15: Phase 1 cost plan against the 6.20 million US dollar limit.

It is worth pointing out clearly that software building is only about 8 percent of the total budget. Equipment, network connections, training and trust building together make up more than half. This is the correct shape for a project like this, because an excellent app running on a clinic with no working sensor, and used by a worker who does not trust it, delivers nothing at all.

5.9 Main Risks to the Estimate

Risk	Why it is a real risk here	How the plan protects against it
The old system turns out to be far more complex than expected	It is a 2011 system with little or no documentation, and no one has confirmed if a test copy even exists	The estimate for this part is treated as provisional until Stage 0 finishes real research on it. A manual backup export is kept as a fallback in case the automatic link cannot be built in time
Health workers reject the app due to trust issues	This has already happened once with the smart thermometer pilot, so it is a proven risk, not a guess	A safety margin of extra time and effort is kept aside for redesign work. The trust programme runs continuously through all twelve months, not just at the start
The NBR customs link takes far longer than expected due to politics	This depends on another government authority and cannot be controlled or sped up by this project	This work is kept off the main critical path completely. The offline paper export is the only committed feature, and the live automatic link is left for a later phase
The size estimate itself turns out to be wrong	Function point counting still involves some judgement calls, so the final size could be higher or lower than expected	The estimate is shown as a range rather than a single fixed number, and it is checked again and updated at every stage once real progress data is available

Table 16: The biggest risks to this estimate and how the plan protects against them.

5.10 A Range of Possible Outcomes

Because small changes in size lead to larger changes in effort, it is more honest to show a range of outcomes rather than one single number.

Scenario	Size (thousand lines of code)	Effort needed (person months)	Fits in 12 months?
Smallest safe core only	9	64	Yes, with room to spare
Recommended Phase 1 core	12	87	Yes, but only just, with some schedule compression
Core plus both major risks happening	15	111	Only just, and would need cuts partway through
Full original scope from the brief	22	168	No, this does not fit in 12 months at all

Table 17: Effort needed for different possible project sizes.

The practical lesson from this table is that the recommended Phase 1 core fits, but only just, with no spare room if two risks happen at the same time. This is exactly why the checkpoint at the end of Stage 1, described in Section 3.7, is treated as a real decision point. If early progress shows the project is heading toward the more difficult end of this range, features can be trimmed early while there is still time to do it in an orderly way, rather than being forced to cut corners in the final month.

6. Conclusion

MeghChain is not really a software problem on its own. It is a public health and trust problem that happens to need software as part of its answer. The method chosen in Section 2 exists because the project has two needs that pull against each other. The funder needs a plan agreed ahead of time. The real conditions in the corridor only become clear once work has already started. Moving the planning checkpoints to every three months lets the project serve both needs at once. The scoring system in Section 3 exists because a plain priority list cannot show that a quiet but essential feature, like keeping a safe audit trail, matters more than a feature that simply looks impressive on a screen. The design in Section 4 pushes its most important safety rules, such as never overwriting a safety record, directly into the structure of the system rather than leaving them as a policy someone has to remember to follow. The estimate in Section 5 gives an honest answer, which is that the full scope described in the brief does not fit into twelve months, and it is more useful to say this clearly with a smaller working plan than to promise something that cannot realistically be delivered.

Two simple tests were set at the very start of this report. Would this design have caught the March 2026 problem? Yes. Detection and blocking of unsafe stock both happen fully on the phone with no need for any internet connection, and the 72 hour safety rule makes sure that a clinic with no signal is never wrongly treated as approved. Would health workers actually be willing to use it? This design gives a strong, honest attempt at yes, through simple location tracking, role based rather than personal identification in daily use, and a friendly, non blaming way of showing sync problems. Whether this is enough in practice is something that can only be proven by testing it with real health workers, which is exactly why the plan puts a working version in front of real clinics as early as Month 6.

7. References

- Boehm, B.W. (1988) 'A Spiral Model of Software Development and Enhancement', IEEE Computer, 21(5), pp. 61 to 72.
- Boehm, B.W., Abts, C., Brown, A.W., Chulani, S., Clark, B.K., Horowitz, E., Madachy, R., Reifer, D.J. and Steece, B. (2000) Software Cost Estimation with COCOMO II. Upper Saddle River, NJ: Prentice Hall.
- Evans, E. (2003) Domain Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison Wesley.
- Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston, MA: Addison Wesley.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995) Design Patterns: Elements of Reusable Object Oriented Software. Reading, MA: Addison Wesley.
- International Function Point Users Group (2010) Function Point Counting Practices Manual, Release 4.3.1. Princeton Junction, NJ: IFPUG.
- ISO/IEC/IEEE 29119 4:2021 Software and Systems Engineering, Software Testing, Part 4: Test Techniques. Geneva: ISO.
- Object Management Group (2017) OMG Unified Modeling Language, Version 2.5.1. Needham, MA: OMG.
- Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. 9th edn. New York: McGraw Hill.
- Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.
- World Health Organization (2015) Immunization in Practice: A Practical Guide for Health Staff. Geneva: WHO.
- World Health Organization (2020) WHO Technical Specifications for Vaccine Cold Chain Equipment and Temperature Monitoring Devices. Geneva: WHO.